

Control Code Dispatch Table Analysis

Function Overview

The `control_code_dispatch_table` function implements a **dual dispatch system** for processing control characters and high characters using jump tables.

Assembly Analysis

Complete Function Disassembly

```
assembly

; Entry point for high characters (0xB0-0xBF with lower 4 bits)
ram:25a0: MOV B,A          ; Save original character in B
ram:25a1: ANI 0xf          ; Mask to lower 4 bits (0x00-0x0F)
ram:25a3: JMP 0x25a9       ; Jump to common dispatch code

; Entry point for control characters (0x00-0x1F)
ram:25a6: MOV B,A          ; Save original character in B
ram:25a7: ANI 0x1f        ; Mask to 5 bits (0x00-0x1F)

; Common dispatch calculation
ram:25a9: ADD A            ; Multiply index by 2 (A = A * 2)
ram:25aa: MOV E,A          ; E = table offset
ram:25ab: MVI D,0x0        ; D = 0 (16-bit offset in DE)
ram:25ad: DAD HL,DE        ; HL = table_base + offset
ram:25ae: MOV E,(HL)       ; Load low byte of function address
ram:25af: INX HL           ; Next byte
ram:25b0: MOV D,(HL)       ; Load high byte of function address
ram:25b1: XCHG             ; HL = function address
ram:25b2: MOV A,B          ; Restore original character
ram:25b3: PCHL             ; Jump to handler function (indirect call)
```

Dispatch Table Structure

Table 1: Control Character Dispatch Table at 0x2421

- **Size:** 32 entries × 2 bytes = 64 bytes
- **Range:** Control characters 0x00-0x1F
- **Entry Point:** 0x25a6 (mask 0x1F)
- **Usage:** Standard ASCII control characters

Table 2: High Character Dispatch Table at 0x238b

- **Size:** 16 entries × 2 bytes = 32 bytes
- **Range:** High characters with lower 4 bits (0x00-0x0F)
- **Entry Point:** 0x25a0 (mask 0x0F)
- **Usage:** Extended/special characters ≥ 0xB0

Control Character Dispatch Table (0x2421)

Based on analysis of the code, the table entries are:

Index	Char	Handler Address	Function
0x00	NUL	0x29b3	Null character handler
0x01	SOH	0x29b9	Start of heading
0x02	STX	0x29bf	Start of text
0x03	ETX	0x29xx	End of text
0x04	EOT	0x29xx	End of transmission
0x05	ENQ	0x29xx	Enquiry
0x06	ACK	0x29xx	Acknowledge
0x07	BEL	0x29xx	Bell
0x08	BS	0x29xx	Backspace
0x09	HT	0x29xx	Horizontal tab
0x0A	LF	0x29db	Line feed → advance_cursor_position
0x0B	VT	0x29xx	Vertical tab
0x0C	FF	0x29xx	Form feed
0x0D	CR	0x29bf	Carriage return → reset_cursor_to_line_start
0x0E	SO	0x29xx	Shift out
0x0F	SI	0x29xx	Shift in
0x10-0x1F	...	0x2xxx	Additional control chars

Handler Function Examples

CR Handler (0x29bf)

assembly

```
ram:29bf: CALL 0x006a      ; Call reset_cursor_to_line_start
ram:29c2: CALL 0x0097      ; Additional cursor operations
ram:29c5: LDA (0x5f24)      ; Load display mode flag
ram:29c8: CPI 0x2          ; Check if mode 2
ram:29ca: JZ 0x29d3        ; Jump if mode 2
ram:29cd: CALL 0x0106      ; Standard cursor update
ram:29d0: JMP 0x20c9       ; Return to main loop
```

LF Handler (0x29db)

assembly

```
ram:29db: CALL 0x0076      ; Call advance_cursor_position
ram:29de: MVI L,0x4e       ; Set column limit
ram:29e0: MVI A,0x20       ; Space character
ram:29e2: CALL 0x0145      ; Clear to end of line
ram:29e5: DCR L            ; Decrement count
ram:29e6: DCR L            ; Decrement again
ram:29e7: JNZ 0x29e2       ; Loop until done
ram:29ea: JMP 0x20c9       ; Return to main loop
```

Call Context Analysis

Control Character Path

assembly

```
; From execute_control_character (0x241b)
ram:241b: LXI HL,0x2421    ; Load control char table address
ram:241e: JMP 0x25a6       ; Jump to control dispatch
```

High Character Path

assembly

```
; From special_character_processor (0x2377)
ram:2377: LXI HL,0x238b    ; Load high char table address
ram:237a: JMP 0x25a0       ; Jump to high char dispatch
```

Memory Data Type Definitions

To properly analyze this in Ghidra, set these data types:

Control Character Table (0x2421)

c

```
// Define as array of function pointers
typedef void (*control_handler_func_t)(byte character);
control_handler_func_t control_char_dispatch_table[32] @ 0x2421;
```

High Character Table (0x238b)

c

```
// Define as array of function pointers
typedef void (*high_char_handler_func_t)(byte character);
high_char_handler_func_t high_char_dispatch_table[16] @ 0x238b;
```

Function Signature

c

```
void control_code_dispatch_table(byte character, void** table_address)
{
    byte index;
    void (*handler)(byte);

    if (table_address == 0x2421) {
        index = character & 0x1F; // Control chars 0x00-0x1F
    } else {
        index = character & 0x0F; // High chars, lower 4 bits
    }

    handler = table_address[index];
    handler(character);
}
```

CR/LF Debug Points

For your CR/LF debugging, monitor these specific handlers:

CR (0x0D) Handler

- **Table Entry:** $0x2421 + (0x0D \times 2) = 0x243B$
- **Handler Address:** Should contain 0x29bf
- **Function:** Calls reset_cursor_to_line_start (0x006a)

LF (0x0A) Handler

- **Table Entry:** $0x2421 + (0x0A \times 2) = 0x2435$

- **Handler Address:** Should contain 0x29db
- **Function:** Calls `advance_cursor_position` (0x0076)

Ghidra Setup Commands

To properly set up the analysis in Ghidra:

1. Set table data types:

- Go to 0x2421, create pointer array[32]
- Go to 0x238b, create pointer array[16]

2. Mark function pointers:

- Each entry should be defined as a code pointer
- Creates cross-references to handler functions

3. Add structure definitions:

- Define the dispatch function signature
- Add parameter types for character handlers

This will give you proper cross-referencing and make it easy to see which characters map to which handlers, especially for debugging your CR/LF issue.